

Create Schema and Models

Overview

In the DarshanEase application, **Mongoose** is used to define schemas and models for MongoDB collections. A **schema** defines the structure and validation rules for the data, while a **model** provides an interface to perform Create, Read, Update, and Delete (CRUD) operations on the corresponding collection.

The application contains five main models:

- User Model
- Temple Model
- Slot Model
- Booking Model
- Admin Model

Project Structure

```
backend/  
├── models/  
│   ├── User.js  
│   ├── Temple.js  
│   ├── Slot.js  
│   ├── Booking.js  
│   └── Admin.js
```

1. User Model

File: models/User.js

```
const mongoose = require("mongoose");  
  
const userSchema = new mongoose.Schema({  
  name: {  
    type: String,  
    required: true  
  },  
  email: {  
    type: String,
```

```

    required: true,
    unique: true
  },
  phone: {
    type: String,
    required: true
  },
  password: {
    type: String,
    required: true
  },
  role: {
    type: String,
    default: "user"
  }
}, {
  timestamps: true
});

module.exports = mongoose.model("User", userSchema);

```

Explanation

- Stores user account details.
- Email is unique for each user.
- Password is stored after hashing.
- timestamps automatically adds createdAt and updatedAt.

2. Temple Model

File: models/Temple.js

```

const mongoose = require("mongoose");

const templeSchema = new mongoose.Schema({
  templeName: {
    type: String,
    required: true
  },
  location: {
    type: String,
    required: true
  },
});

```

```
description: String,  
openingTime: String,  
closingTime: String  
}, {  
  timestamps: true  
});
```

```
module.exports = mongoose.model("Temple", templeSchema);
```

Explanation

- Stores temple information.
- Used to display temples in the application.
- Linked with darshan slots.

3. Slot Model

File: models/Slot.js

```
const mongoose = require("mongoose");
```

```
const slotSchema = new mongoose.Schema({  
  templeId: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: "Temple",  
    required: true  
  },  
  date: {  
    type: Date,  
    required: true  
  },  
  startTime: String,  
  endTime: String,  
  availableSeats: {  
    type: Number,  
    default: 100  
  }  
}, {  
  timestamps: true  
});
```

```
module.exports = mongoose.model("Slot", slotSchema);
```

Explanation

- Represents available darshan slots.
- References the Temple collection using ObjectId.
- Tracks seat availability.

4. Booking Model

File: models/Booking.js

```
const mongoose = require("mongoose");

const bookingSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },
  slotId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Slot",
    required: true
  },
  numberOfPersons: {
    type: Number,
    default: 1
  },
  bookingStatus: {
    type: String,
    default: "Confirmed"
  }
}, {
  timestamps: true
});

module.exports = mongoose.model("Booking", bookingSchema);
```

Explanation

- Stores booking information.
- References both the User and Slot collections.
- Maintains booking status and number of persons.

5. Admin Model

File: models/Admin.js

```
const mongoose = require("mongoose");

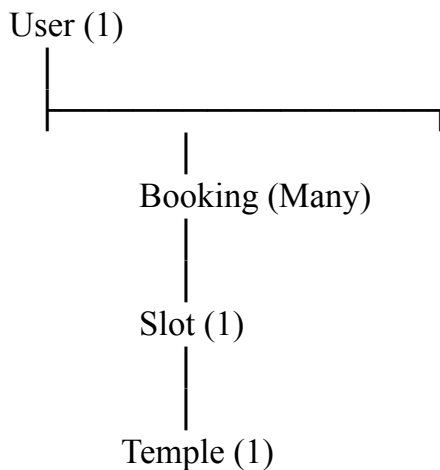
const adminSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    unique: true,
    required: true
  },
  password: {
    type: String,
    required: true
  }
}, {
  timestamps: true
});

module.exports = mongoose.model("Admin", adminSchema);
```

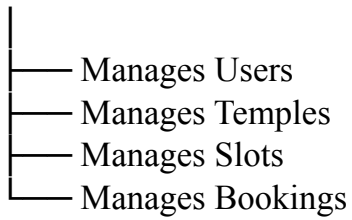
Explanation

- Stores administrator credentials.
- Used for secure access to the admin dashboard.

Model Relationships



Admin



Benefits of Using Mongoose Models

- Defines a clear structure for application data.
- Enforces validation rules before saving documents.
- Simplifies CRUD operations.
- Supports relationships using ObjectId references.
- Automatically manages timestamps.
- Improves code organization and maintainability.

Conclusion

The Mongoose schemas and models provide the foundation for the DarshanEase application's data layer. By defining structured collections for users, temples, slots, bookings, and administrators, the system ensures data consistency, efficient database operations, and easy integration with the application's business logic through the MVC architecture.